When an MCP Benchmark's Data and Evaluator Disagree: A Pinned Artifact Audit

Ezekiel Ologunde

ABSTRACT

Security benchmark results depend on consistent interpretation of labels and inputs. We audit seven separate exports of AgentDefense-Bench at one pinned revision. On its documented 640-record tier-1 evaluation path, the evaluator ignores Boolean `is_benign` labels and defaults to an allowed expectation. An unchanged mock detector blocks 39 records: these are counted as false positives by the original evaluator and as true positives after explicit label normalization. All paired adapter decisions remain identical. A separate README example selects an absent JSON array and iterates over zero cases. We provide a locally verified acquisition and replay package. These findings demonstrate artifact contract failures; they establish neither production defense efficacy nor effects on published model rankings. The contribution is a bounded reproducibility audit, not a new defense or a guarantee of first discovery.

KEYWORDS

MCP, benchmark validity, evaluation, reproducibility, label schema

1 INTRODUCTION AND QUESTIONS

AgentDefense-Bench provides MCP-shaped inputs and adapters for evaluating security defenses [1]. Its evaluation scripts must agree with the schemas of the files recommended by its documentation. Otherwise, a correctly transported input can still receive an incorrect evaluation interpretation.

We ask three questions. RQ1: Does the documented evaluator interpret the named export's declared labels consistently? RQ2: Does the programmatic example select the available records? RQ3: How do confusion counts change when only label representation is normalized? These questions concern a specific artifact rather than the prevalence of benchmark defects across the field.

We provide a pinned label trace, paired reproduction and replay supplement. This technical report, prepared on 21 October 2026, is not peer reviewed.

2 ARTIFACT AND PROTOCOL

We pin `arunsanna/AgentDefense-Bench` at revision `b5dfdf3c`… (full identifier in the input manifest). Fourteen selected files are identified by byte size, Git blob SHA-1 and SHA-256. They total 46,306,066 bytes. Acquisition fetches Git objects without checkout or submodules; payload strings remain inert data.

Source and schema inspection preceded the protocol. The observed label branches suggested that records labelled only through `is_benign` would reach the default-allowed branch, whereas explicit expected fields would retain their interpretation. This is an exploratory expectation formed after inspection, not a blind or preregistered hypothesis.

Seven exports are evaluated separately. The primary case is the tier-1 CLI path explicitly recommended in `docs/INTEGRATION.md`. The other exports are secondary schema-transfer probes. We do not pool them or treat repeated/overlapping exports as independent samples.

The harness selects inspected upstream definitions through Python's abstract syntax tree: `MockDefense`, its required base/result classes, `load_dataset`, `run_evaluation`, and `EvaluationResult`. Their bodies are unchanged. Network adapter factories and the full CLI entry point are excluded. The mock serializes requests as JSON and searches for predefined strings; no payload is executed.

The comparator adds `expected_block = not is_benign` only where the latter is Boolean. All other labels, identifiers and requests remain unchanged. Temporary ASCII-escaped JSON preserves parsed strings while avoiding the upstream loader's Windows default-encoding dependency. We require identical paired adapter decision vectors and call counts. The comparator is an established schema-normalization technique, not a novel defense.

3 CONTRACT FAILURES

3.1 Label Projection

The evaluator recognizes `expected_block`, `expected_outcome`, `expected`, or the literal category `attack`. It does not recognize `is_benign`. The tier files instead supply Boolean `is_benign` and specific category names. Their attack-labelled rows therefore reach the default-allowed branch.

Across the three tier files, 33,118 records have `is_benign=false`; 250 further tier-3 records have `is_benign=true`. All 33,118 attack-labelled records receive an allowed expectation. This is a mismatch with declared labels, not independent adjudication that every record is malicious.

3.2 Record Selection and Request Availability

The README's programmatic example opens the tier-1 file but selects `get('attacks', [])`. That file provides `tests`, so the example selects zero rows despite 640 available records. We evaluate the selector only; the example's HTTP adapter is not instantiated.

The separate 250-record consolidated benign export lacks both request fields selected by the evaluator, producing 250 empty dictionaries. It differs from the request-bearing 93-record benign baseline. Tier 3 also supplies three empty selected requests. A reported successful benign decision on an empty input is not evidence of correct classification of the omitted prompt.

4 PAIRED RESULTS

Table 1 reports exact counts. TP, FP, TN and FN are the evaluator's confusion categories relative to the respective label interpretation. They do not independently validate the source's natural labels.

On the documented tier-1 path, the original evaluator records TP=0, FP=39, TN=601 and FN=0. After normalization, counts are TP=39, FP=0, TN=0 and FN=601. The detector makes exactly the

Table 1: Separate export-level results with unchanged upstream mock decisions. Normalization affects labels only. Counts are not pooled.

Export	Records	Empty requests	Original TP/FP/TN/FN	Normalized TP/FP/TN/FN	Role
Combined attacks	319	0	65/0/0/254	65/0/0/254	Secondary
Novel export	100	0	17/0/15/68	17/0/15/68	Secondary
Benign baseline	93	0	0/0/93/0	0/0/93/0	Secondary
Tier 1	640	0	0/39/601/0	39/0/0/601	Primary
Tier 2	27,122	0	0/30/27,092/0	30/0/0/27,092	Secondary
Tier 3	5,606	3	0/102/5,504/0	102/0/250/5,254	Secondary
Separate benign	250	250	0/0/250/0	0/0/250/0	Secondary

same 39 blocked decisions in both runs. The difference concerns score semantics, not improved detector behavior.

The combined, novel and request-bearing benign exports retain their original counts. Tier 3 includes benign-labelled rows; treating all tier records as attacks would overstate the mismatch by 250. The initial machine-readable qualification retained the correct Boolean counts; a preliminary prose total was corrected during paired reproduction.

The README advertises 35,989 cases. Our named frames have distinct schemas and unresolved overlap/generation history. Comparing one selected frame to that total does not establish an incorrect advertised count. Separately, the published checksum for the novel export does not match its pinned bytes, although acquisition identities do verify. A stale checksum is possible; tampering is not established.

5 HISTORY AND RELATED WORK

The current default-branch path history returned two evaluator commits and one integration-guide commit in a first-page query capped at 100. The evaluator fetched from the December 2025 integration commit `f196a8e0`… is byte-identical to the pinned April 2026 version. This confirms persistence across the inspected versions, not every branch or fork. Repository issue search returned one complementary-benchmark cross-reference; its body does not discuss these failures. Comments and external discussion were not exhaustively searched. No maintainer was contacted.

Existing `mcp-defense-bench` positioning distinguishes coverage completeness from AgentDefense-Bench detection accuracy and already questions repackaging general safety prompts as MCP requests [2]. Those general criticisms are prior work, not our contribution. The referenced positioning page was inspected on mutable `main`; its broader assertions are not independently validated here.

MCP-Guard reports AgentDefense external evaluation in Section 5.2 and Table 4b [3]. The inspected sections do not establish that those scores use this pinned evaluator or tier-1 path. Our experiment therefore does not invalidate its results. Matching source versions, prediction logs and evaluation paths would be required to establish published-score effects.

Targeted searches did not establish an earlier reproduction of this mismatch; search absence is not proof of novelty.

6 REPRODUCTION AND LIMITATIONS

A clean extracted package completed three commands under CPython 3.12.14 and Git: restoration, paired audit, and exact-result verification. Restoration checked all 14 files. The audit preserved ten input source/data/protocol SHA-256 values and identical paired decisions in all frames. The complete result JSON matched its retained expected result. The final archive updates documentation and includes the verification receipt; its executable inputs are byte-identical to the tested extraction.

No model, external endpoint or production defense was evaluated. AST-selected execution reproduces the relevant function bodies, not full CLI startup. The mock is an interface example, not a security baseline against which to claim deployment readiness. Source labels remain independently unvalidated. No confidence interval or population inference is warranted from deterministic artifact recounts.

Thirty configuration-drift-labelled requests exist in tier 1, but their schema does not supply validated before/after authority states or temporal permission histories. This audit does not solve longitudinal authority drift, credential blast radius or delegation correctness.

7 AVAILABILITY AND STATUS

The standalone supplement contains authored code, protocol, hash manifests, aggregate results and restoration instructions. It restores raw multi-source data and upstream code from the pinned origin rather than redistributing them under a blanket licence. Source-specific terms remain applicable. Authored reuse terms have not been selected; rights are reserved. The draft and supplement were locally verified and have not yet been added to the public report collection. Independent review remains pending.

REFERENCES

[1] Arun Sanna. 2025. AgentDefense-Bench. Pinned revision b5dfdf3c (full identifier in supplement). https://github.com/arunsanna/AgentDefense-Bench.

[2] Gowthaman90. 2026. mcp-defense-bench: Prior Art and Novelty Positioning. Mutable main-branch page inspected 21 October 2026. https://github.com/Gowthaman90/mcp-defense-bench/blob/main/docs/PRIOR-ART.md.

[3] Wenpeng Xing et al. 2026. MCP-Guard: A Multi-Stage Defense-in-Depth Framework for Securing Model Context Protocol in Agentic AI. In *Findings of ACL 2026*, 4877–4889. https://doi.org/10.18653/v1/2026.findings-acl.240.